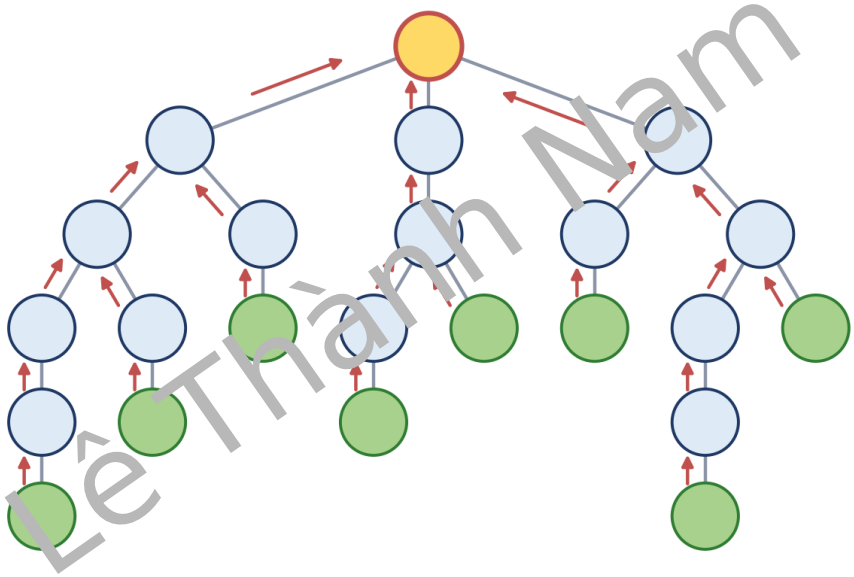
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

THUẬT TOÁN TRÊN CÂY

Dễ hiểu cho người bắt đầu từ con số 0

TẬP 3 · QUY HOẠCH ĐỘNG TRÊN CÂY



Lê Thành Nam

Tập độc lập · Đổi gốc · Cái túi trên cây · Chuỗi dài · Li Chao có hoàn tác · Slope trick

Chương 1. QUY HOẠCH ĐỘNG CƠ BẢN TRÊN CÂY
Chương 2. ĐỔI GỐC (REROOTING)
Chương 3. BÀI TOÁN CÁI TÚI TRÊN CÂY
Chương 4. QUY HOẠCH ĐỘNG THEO ĐỘ SÂU
Chương 5. BAO LỒI VÀ CÂY LI CHAO TRÊN CÂY
Chương 6. SLOPE TRICK VÀ HEAP GỘP ĐƯỢC

LỜI NÓI ĐẦU

Quy hoạch động (QHĐ) là kĩ thuật xuất hiện nhiều nhất trong các đề thi học sinh giỏi Tin học, và cây là nơi QHĐ "sống" tự nhiên nhất: mỗi cây con là một bài toán con, và bỏ một đỉnh đi thì các cây con tách rời nhau. Từ bài toán chọn khách mời cho bữa tiệc đến bài pháo hoa của APIO, hầu như mọi bài QHĐ trên cây đều đi theo cùng một khuôn: chọn đúng **trạng thái**, rồi **gộp** các cây con.

Cuốn sách này là **Tập 3: Quy hoạch động trên cây** của bộ sách về thuật toán trên cây, tiếp nối Tập 1 (Nền tảng) và Tập 2 (Tổ tiên chung gần nhất). Em cần đã quen với: danh sách kề, mảng cha, BFS và thứ tự BFS, DFS không đệ quy, kích thước cây con, LCA. Những kiến thức ấy được nhắc lại ngắn gọn ở Phần mở đầu.

Giống hai tập trước, mỗi bài được chia làm hai phần:

Phần A – Lí thuyết: định nghĩa, định lí kèm chứng minh từng ý, thuật toán, chạy tay từng bước trên hình (phần lớn trên cây mẫu T14), code mẫu, độ phức tạp, bẫy thường gặp.

Phần B – Bài vận dụng: một bài toán theo khuôn đề thi, giải đầy đủ: ý tưởng ngây thơ với sơ đồ khối và chương trình vét cạn, chỉ ra bế tắc, áp dụng lí thuyết để phá bế tắc, chạy tay, chương trình hoàn chỉnh, mở rộng và luyện tập.

Sách gồm **Phần mở đầu** (vì sao QHĐ hợp với cây, khuôn 5 bước, các kiểu trạng thái, bản đồ tập sách) và **15 bài học chia thành 6 chương**, độ khó tăng dần từ cơ bản tới nâng cao:

Chương 1. QHĐ cơ bản: tập độc lập, phủ đỉnh, tập thống trị, cặp ghép trên cây.

Chương 2. Đổi gốc: đáp án cho mọi gốc trong O(n), với tổng và với max.

Chương 3. Cái túi trên cây: gộp con theo kích thước, chứng minh $O(n^2)$ và $O(n \cdot k)$.

Chương 4. QHĐ theo độ sâu: mảng độ sâu, phân rã chuỗi dài O(n).

Chương 5. Bao lồi trên cây: cây Li Chao có hoàn tác, QHĐ trên đường về gốc.

Chương 6. Slope trick: hàm lồi tuyến tính từng khúc, heap trái gộp được.

Các bài vận dụng gồm những bài kinh điển: CSES Tree Matching (Bài 4), CSES Tree Distances I, II (Bài 5, 6), IOI 2005 Rivers (Bài 8), Codeforces 1009F Dominant Indices (Bài 10), CEOI 2009 Harbingers (Bài 12), APIO 2012 Dispatching (Bài 14), APIO 2016 Fireworks (Bài 15), phần lớn được đặt lại trong bối cảnh nhà trường. Mọi chương trình đều đã được đối chiếu với chương trình vét cạn trên hàng trăm cây ngẫu nhiên, và chạy thử với dữ liệu lớn nhất; mọi ví dụ trong sách đều được chạy lại bằng chính các chương trình đó.

Chương trình viết bằng C++17, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp", dùng được với Themis lẫn các trang chấm trực tuyến. Mọi chương trình đều không có chú thích bên trong; các dòng quan trọng được giải thích ngay bên dưới.

Chúc các em và các thầy cô học tập say mê, dạy tốt và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Phần	Nội dung
Tóm tắt bài học	Khung tóm tắt ở đầu bài: kiến thức chính, khuôn cài đặt, độ phức tạp, khi nào dùng, bài vận dụng.
Phần A – Lí thuyết	Định nghĩa, tính chất, định lí kèm chứng minh từng ý; thuật toán; chạy tay từng bước nhỏ trên hình; code mẫu; độ phức tạp; bẫy; ghi nhớ.
Phần B – Bài vận dụng	Đề bài như đề thi thật: dữ liệu vào, kết quả, giới hạn, ví dụ.
B1. Ý tưởng ngây thơ	Cách làm ai cũng nghĩ ra: sơ đồ khối, chạy tay, chương trình vét cạn, đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ quá chậm hoặc quá tốn bộ nhớ.
B2. Phá bế tắc	Áp dụng lí thuyết Phần A, từng bước, chạy tay trên hình.
B3. Chương trình hoàn chỉnh	Chương trình C++ đã kiểm chứng, giải thích các dòng quan trọng.
B4. Mở rộng, Luyện tập	Các biến thể thường gặp; bài luyện thêm và cách tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
TÓM TẮT BÀI HỌC	Tóm tắt toàn bộ bài ở ngay đầu bài.
ĐỊNH NGHĨA	Định nghĩa chính xác một khái niệm.
TÍNH CHẤT / ĐỊNH LÍ	Một khẳng định quan trọng, luôn có chứng minh đi kèm.
THUẬT TOÁN	Các bước của một thuật toán.
HỘP CÔNG CỤ C++	Giới thiệu một công cụ C++ khi nó được dùng lần đầu.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp: vì sao chậm, vì sao nhanh.
BẾ TẮC	Chỗ cách làm ngây thơ không vượt qua được.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối phần lí thuyết.

Quy ước màu trong các hình được giữ nguyên trong cả cuốn sách: **đỉnh vàng viền đỏ** là đỉnh đang xét; **đỉnh xanh lá** là đỉnh đã xử lí xong, hoặc đỉnh được chọn trong lời giải; **đỉnh xanh dương nhạt** là đỉnh bình thường; **đỉnh đỏ nhạt** là đỉnh bị loại hoặc vi phạm điều kiện; **đỉnh tím** là tổ tiên đang dùng; **gốc** tô xanh đậm. Số nhỏ cạnh mỗi đỉnh là giá trị QHĐ đang tính (chẳng hạn f0 | f1); số trên cạnh là trọng số của cạnh; mũi tên dọc cạnh chỉ hướng truyền thông tin.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Gặp một hình chạy tay, hãy che phần lời giải lại, tự làm bước đó rồi mới đọc. Gặp một định lí, hãy tự vẽ một cái cây khác và kiểm tra định lí trên cây của mình. Chỉ khi tự viết lại được chương trình và tự giải thích được vì sao nó đúng mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**
CÁCH HỌC VỚI TÀI LIỆU NÀY **3**
PHẦN MỞ ĐẦU. QUY HOẠCH ĐỘNG TRÊN CÂY VÀ BẢN ĐỒ TẬP 3 ..... **6**
CHƯƠNG 1. QUY HOẠCH ĐỘNG CƠ BẢN TRÊN CÂY **11**
Bài 1. Tập độc lập lớn nhất 12
Bài 2. Phủ đỉnh nhỏ nhất 29
Bài 3. Tập thống trị 44
Bài 4. Cặp ghép trên cây 57
CHƯƠNG 2. ĐỔI GỐC (REROOTING) **68**
Bài 5. Đổi gốc: tổng khoảng cách 69
Bài 6. Đổi gốc với max: đỉnh xa nhất 86
CHƯƠNG 3. BÀI TOÁN CÁI TÚI TRÊN CÂY **102**
Bài 7. Cái túi trên cây 103
Bài 8. Cái túi với trạng thái phụ 116
CHƯƠNG 4. QUY HOẠCH ĐỘNG THEO ĐỘ SÂU **128**
Bài 9. QHĐ theo độ sâu 129
Bài 10. Phân rã chuỗi dài 143
CHƯƠNG 5. BAO LỒI VÀ CÂY LI CHAO TRÊN CÂY **158**
Bài 11. Cây Li Chao có hoàn tác 159
Bài 12. QHĐ trên đường về gốc với Li Chao 173
CHƯƠNG 6. SLOPE TRICK VÀ HEAP GỘP ĐƯỢC **186**
Bài 13. Slope trick 187
Bài 14. Heap trái gộp được 196
Bài 15. Slope trick trên cây 208
TỔNG KẾT: NHẬN DẠNG BÀI QUY HOẠCH ĐỘNG TRÊN CÂY **220**

Lê Thành Nam

PHẦN MỞ ĐẦU

QUY HOẠCH ĐỘNG TRÊN CÂY VÀ BẢN ĐỒ TẬP 3

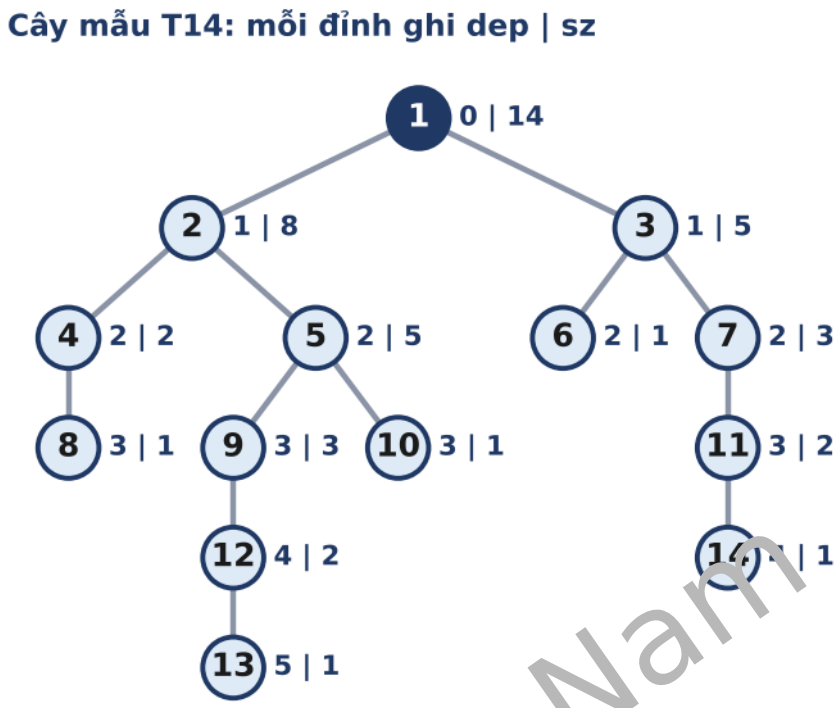
Phần này nhắc lại những gì cần từ Tập 1 và Tập 2, giải thích vì sao quy hoạch động (QHĐ) đặc biệt hợp với cây, đưa ra khuôn 5 bước dùng cho cả cuốn sách, phân loại các kiểu trạng thái, và vẽ bản đồ sáu chương.

1. Nhắc lại từ Tập 1 và Tập 2

Cây luôn được **treo ở gốc 1** (trừ khi đề cho sẵn gốc khác). Các kí hiệu dùng xuyên suốt:

Kí hiệu	Ý nghĩa	Cách tính
par[v]	cha của v; par[gốc] = 0	đọc trực tiếp, hoặc BFS nếu cho dạng cạnh
ord	thứ tự BFS: cha luôn đứng trước con	BFS bằng chính vector ord
T(v)	cây con gốc v: v và mọi hậu duệ	
dep[v], dist[v]	độ sâu (số cạnh), khoảng cách có trọng số tới gốc	BFS, cha trước con
sz[v]	số đỉnh của T(v)	duyệt ord ngược, sz[par] += sz[con]
h[v]	chiều cao: số đỉnh trên đường dài nhất từ v xuống lá	duyệt ord ngược (Bài 10)
LCA(u, v)	tổ tiên chung gần nhất	Tập 2

Lê Thành Nam



***Hình 0.1.** Cây mẫu T14*

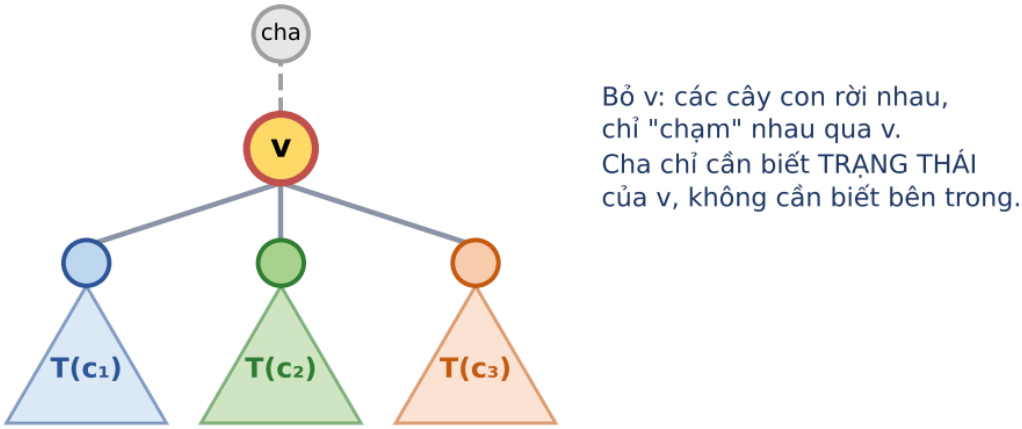
Phần lớn các hình chạy tay dùng cây T14 quen thuộc. Danh sách cạnh của T14 (dùng trong các ví dụ): 1 2, 1 3, 2 4, 2 5, 3 6, 3 7, 4 8, 5 9, 5 10, 7 11, 9 12, 12 13, 11 14. Mảng cha: p_2, ..., p_{14} = 1 1 2 2 3 3 4 5 5 7 9 12 11.

> **KHÔNG ĐỆ QUY**
> Mọi chương trình trong sách duyệt cây **không đệ quy**: QHĐ "con trước cha" dùng **ngược thứ tự BFS**, khi cần thứ tự DFS thật sự (Bài 12) dùng ngăn xếp tự quản lí. Ngoại lệ duy nhất là phép gộp heap trái (Bài 14, 15), có độ sâu đệ quy chỉ O(log n).

2. Vì sao QHĐ hợp với cây?

QHĐ cần hai điều: bài toán lớn chia được thành **bài toán con**, và lời giải tối ưu ghép được từ lời giải tối ưu của bài toán con. Trên cây, hai điều này có sẵn:

Lê Thành Nam



Hình 0.2. *Bỏ một đỉnh, cây con tách rời*

- **Bài toán con tự nhiên:** cây con T(v). Có đúng n bài toán con, và chúng lồng nhau theo quan hệ cha – con.
- **Tách rời:** bỏ v, các T(c) không có cạnh nào nối với nhau. Mọi ràng buộc giữa các mảnh đều đi qua v. Vì thế chỉ cần ghi nhớ một ít thông tin về v (gọi là **trạng thái**), các mảnh được tối ưu riêng rẽ.

Trên đồ thị tổng quát, tập độc lập lớn nhất là bài toán NP-khó; trên cây, Bài 1 giải nó trong O(n). Sự khác biệt nằm ở tính chất tách rời này.

3. Khuôn 5 bước

KHUÔN 5 BƯỚC (CHI TIẾT Ở BÀI 1)

1. **Treo cây** và xác định bài toán con trên T(v).
2. **Trạng thái:** f[v][s], s là thông tin tối thiểu mà phần còn lại của cây cần biết về T(v).
3. **Công thức:** tính f[v][·] từ f[c][·] của các con; chứng minh bằng tính tách rời.
4. **Thứ tự:** con trước cha (duyệt ngược BFS); cơ sở ở lá.
5. **Đáp số** ở gốc; **truy vết** nếu cần.

Câu hỏi then chốt luôn là ở bước 2: **"cha cần biết gì về cây con?"**. Câu trả lời quyết định kiểu trạng thái, và mỗi chương của sách ứng với một kiểu.

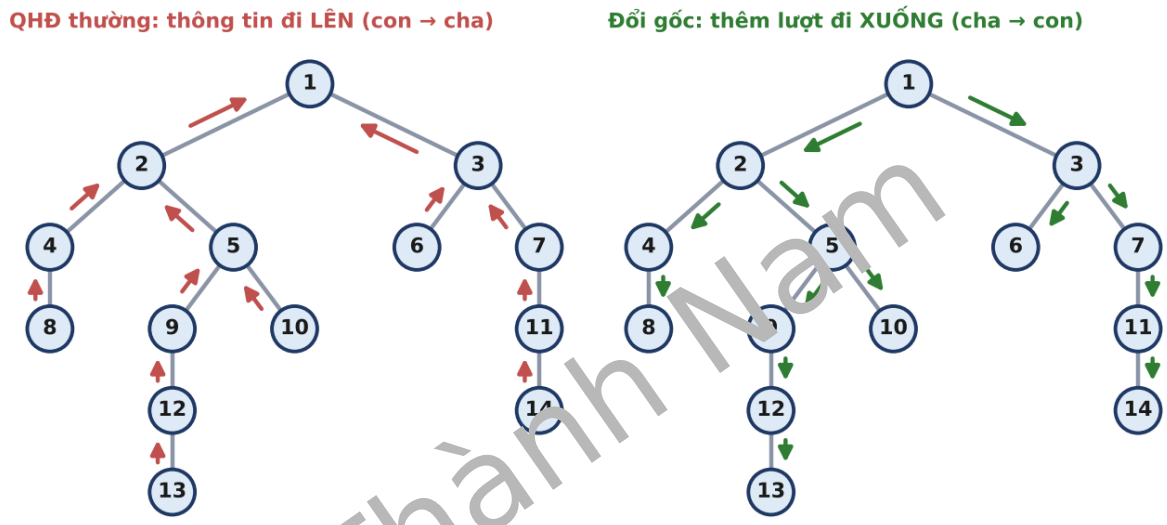
4. Các kiểu trạng thái

Kiểu trạng thái	Kích thước	Gộp con	Chương
vài số (chọn / không chọn, ...)	O(1)	cộng, max, min: O(1)	1
vài số + thông tin "phía trên"	O(1)	thêm lượt đi xuống	2
mảng theo số lượng đã chọn	O(sz) hoặc O(k)	nhân đa thức (max, +)	3

Lê Thành Nam

Kiểu trạng thái	Kích thước	Gộp con	Chương
mảng theo độ sâu	O(h)	cộng mảng dịch 1 ô; kế thừa con dài	4
min của các hàm bậc nhất trên đường về gốc	O(log C) mỗi thao tác	cây Li Chao + hoàn tác	5
hàm lồi tuyến tính từng khúc	số điểm gãy	gộp heap trái	6

5. Hai luồng thông tin

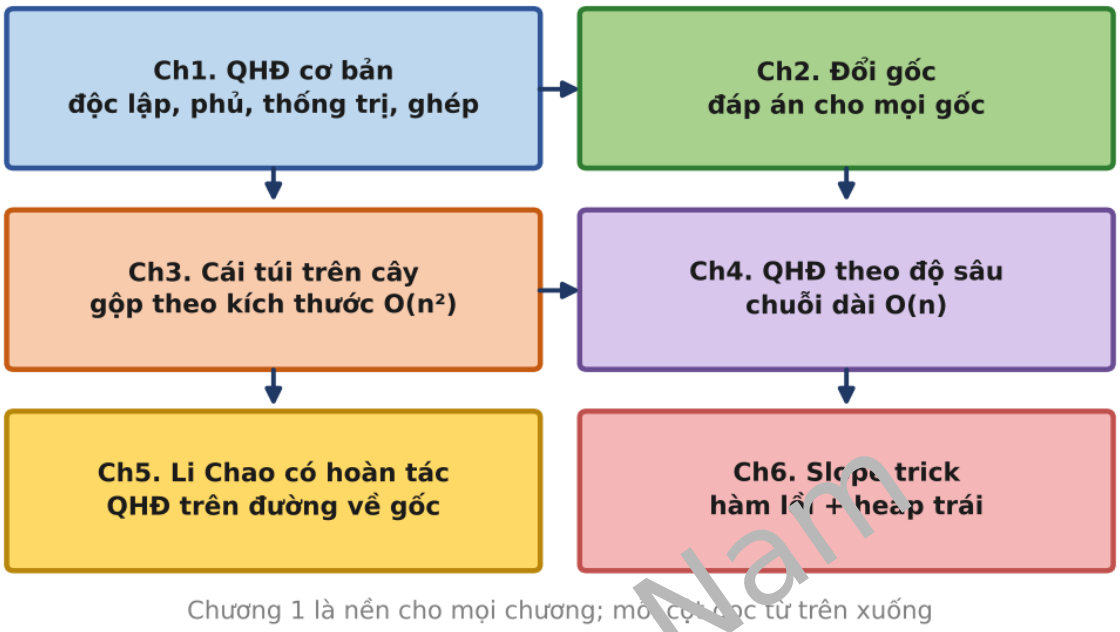


***Hình 0.3.** Thông tin đi lên và đi xuống*

QHĐ cơ bản chỉ cần thông tin **đi lên**: mỗi cây con báo cáo cho cha. Khi cần đáp án cho **mọi** đỉnh làm gốc (Chương 2), hoặc khi đáp án của v phụ thuộc vào **các tổ tiên** của v (Chương 3 Bài 8, Chương 5), thông tin còn phải **đi xuống**: từ cha, mang theo hiểu biết về "phần còn lại của cây".

Lê Thành Nam

6. Bản đồ Tập 3



Hình 0.4. *Sáu chương và thứ tự đọc*

- **Chương 1 (Bài 1–4)** là nền tảng, phải đọc trước. Mỗi bài thêm một kĩ thuật nhỏ: hai trạng thái, phần bù, ba trạng thái với "ít nhất một con", "đúng một con".
- **Chương 2 (Bài 5–6):** đổi gốc, với phép gộp có trừ (tổng) và không có trừ (max).
- **Chương 3 (Bài 7–8):** cái túi trên cây, định lí $O(n^2)$ và $O(n \cdot k)$.
- **Chương 4 (Bài 9–10):** QHĐ theo độ sâu, phân rã chuỗi dài $O(n)$.
- **Chương 5 (Bài 11–12):** cây Li Chao có hoàn tác, QHĐ trên đường về gốc.
- **Chương 6 (Bài 13–15):** slope trick, heap trái, và bài APIO 2016 Fireworks.

Các chương 3–6 độc lập với nhau tương đối; chương 6 cần Bài 13 trước Bài 15, và Bài 14 cho cấu trúc heap.

7. Đối chiếu chương trình

Mỗi bài vận dụng có hai chương trình: chương trình chính (tên bài, chẳng hạn TIECLH.cpp) và chương trình trâu (TIECLH_TRAU.cpp, ghi ra tệp .ANS). Mọi chương trình chính trong sách đã được đối chiếu với chương trình trâu trên 400 cây ngẫu nhiên nhỏ thuộc năm dạng (ngẫu nhiên, đường thẳng, hình sao, "con sâu", cây sâu), và chạy thử với dữ liệu lớn nhất. Em nên tự làm lại việc đối chiếu này cho từng bài: đó là cách tốt nhất để tin vào chương trình của mình.

Lê Thành Nam

CHƯƠNG 1. QUY HOẠCH ĐỘNG CƠ BẢN TRÊN CÂY

Bỏ một đỉnh đi, cây vỡ thành những mảnh rời nhau. Vì thế lời giải tốt nhất của cả cây được ghép từ lời giải tốt nhất của từng cây con, chỉ cần nhớ thêm một chút thông tin ở chỗ nối: đỉnh gốc của mỗi cây con đang ở trạng thái nào.

TRONG CHƯƠNG NÀY

Bài 1. Tập độc lập lớn nhất *(vận dụng: tiệc liên hoan)*

Bài 2. Phủ đỉnh nhỏ nhất *(vận dụng: lắp camera hành lang)*

Bài 3. Tập thống trị *(vận dụng: phủ sóng wifi)*

Bài 4. Cặp ghép trên cây *(vận dụng: bạn cùng bàn (CSES Tree Matching))*

Lê Thành Nam

BÀI 1 Tập độc lập lớn nhất

Bài vận dụng: Tiệc liên hoan

TÓM TẮT BÀI 1. TẬP ĐỘC LẬP LỚN NHẤT

Chủ đề: QHĐ cơ bản trên cây

Kiến thức chính:

- Tập độc lập: không hai đỉnh nào kề nhau; trên cây treo ở gốc: không có cặp cha – con.
- **f1[v]** = a[v] + Σ f0[c]; **f0[v]** = Σ max(f0[c], f1[c]); đáp số max(f0[1], f1[1]).
- Tính con trước cha bằng **duyệt ngược BFS**, đẩy đóng góp lên cha; truy vết từ gốc xuống.
- Khuôn 5 bước: treo cây, trạng thái, công thức, thứ tự, đáp số.

Khuôn cài đặt:

```
for (k = n - 1; k >= 0; k--) {
    u = ord[k]; f1[u] += a[u];
    if (u != 1) {
        f0[par[u]] += max(f0[u], f1[u]);
        f1[par[u]] += f0[u];
    }
}
```

Độ phức tạp: O(n) thời gian và bộ nhớ

Dùng khi: Chọn tập đỉnh, ràng buộc chỉ giữa cha và con

Bài vận dụng: Tiệc liên hoan [TIEC_LH]

Lê Thanh Nam

Học xong bài này, em sẽ:

- ✓ Hiểu thế nào là **quy hoạch động trên cây**: trạng thái gắn với một cây con, đáp án của cây con được ghép từ đáp án của các cây con nhỏ hơn.
- ✓ Giải bài toán **tập độc lập có tổng trọng số lớn nhất** trên cây bằng hai trạng thái f0[v], f1[v]; chứng minh công thức truy hồi.
- ✓ Biết **thứ tự tính** (con trước cha) mà không cần đệ quy: duyệt ngược thứ tự BFS.
- ✓ Biết **truy vết** để in ra tập tối ưu, và nắm **khuôn 5 bước** dùng cho mọi bài QHĐ trên cây trong sách.

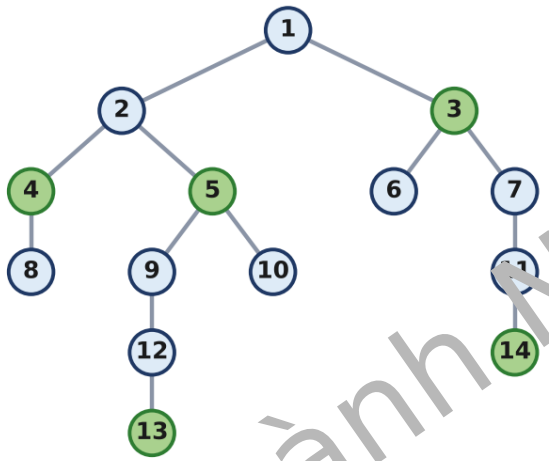
PHẦN A LÍ THUYẾT

A1. Tập độc lập

ĐỊNH NGHĨA

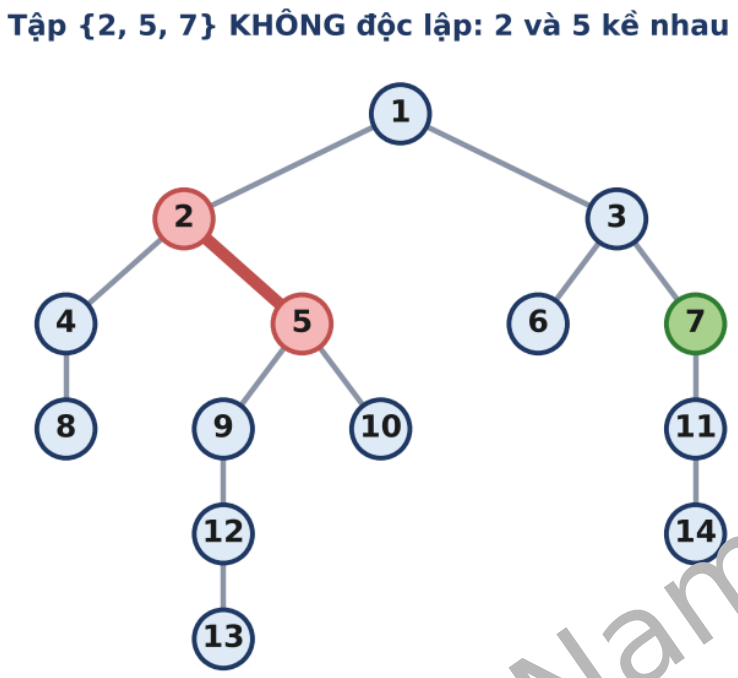
- Cho cây n đỉnh. Một tập đỉnh S gọi là **tập độc lập** nếu **không có hai đỉnh nào của S kề nhau** (không có cạnh nào nối hai đỉnh của S).
- Nếu mỗi đỉnh v có trọng số a[v] ≥ 0 thì **trọng số** của S là tổng a[v] với v ∈ S. Bài toán: tìm tập độc lập có trọng số **lớn nhất**.

Tập {3, 4, 5, 13, 14}: không có hai đỉnh nào kề nhau → tập độc lập



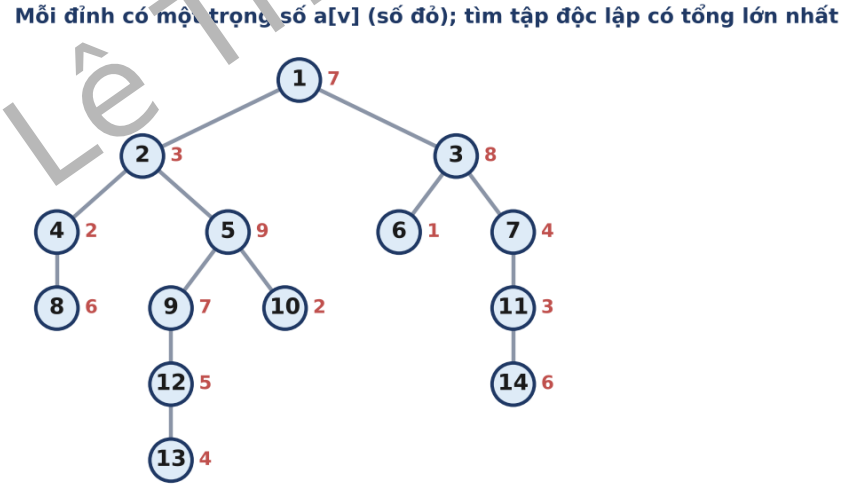
***Hình 1.1** Một tập độc lập của cây T14*

Lê Thành Nam



Hình 1.2. *Một tập không độc lập*

Trên cây đã treo ở gốc 1, hai đỉnh kề nhau luôn là **một cặp cha – con**. Vì vậy điều kiện "độc lập" viết lại là: **không có đỉnh nào cùng thuộc S với cha của nó**. Đây là cách nhìn quan trọng: mọi ràng buộc chỉ nằm giữa một đỉnh và cha của nó.



Hình 1.3. *Cây T14 với trọng số a[v]*

Lê Thành Nam

VÌ SAO KHÔNG THAM LAM?

Ý nghĩ "lấy đỉnh nặng nhất trước" sai: trên đường 3 đỉnh với trọng số 5, 6, 5, tham lam lấy đỉnh giữa (6), trong khi lấy hai đầu được 10.

Ý nghĩ "lấy mọi đỉnh ở tầng chẵn, hoặc mọi đỉnh ở tầng lẻ" (luôn là tập độc lập) cũng sai: với T14, tầng chẵn (0, 2, 4) cho 7 + 16 + 11 = 34, tầng lẻ (1, 3, 5) cho 11 + 18 + 4 = 33, nhưng đáp số đúng là **38**. Ta cần một phương pháp chắc chắn đúng: quy hoạch động.

A2. Trạng thái: hai câu hỏi cho mỗi cây con

Gọi T(v) là **cây con gốc v** (v cùng mọi hậu duệ của v). Ý tưởng QHĐ: giải bài toán cho T(v) khi đã biết lời giải của T(c) với mọi con c của v. Nhưng chỉ biết "giá trị tốt nhất trong T(c)" là **chưa đủ**: nếu v được chọn thì c không được chọn, và ta cần biết lời giải tốt nhất của T(c) **khi c không được chọn**. Vì vậy mỗi đỉnh cần **hai số**.

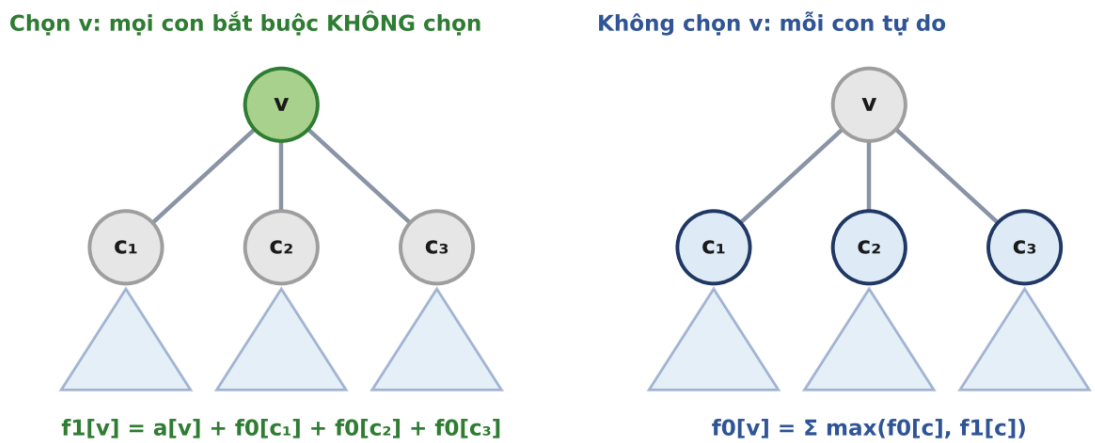
HAI TRẠNG THÁI

- **f1[v]** = trọng số lớn nhất của một tập độc lập S ⊆ T(v) **có chứa v**.
- **f0[v]** = trọng số lớn nhất của một tập độc lập S ⊆ T(v) **không chứa v**.
- Đáp số của bài toán: **max(f0[1], f1[1])**.

TRẠNG THÁI = THÔNG TIN CHA CẦN BIẾT

Câu hỏi để chọn trạng thái: "Khi ghép T(v) vào cha của v, cha cần biết gì về T(v)?" Cha chỉ cần biết v có được chọn hay không, nên hai trạng thái là đủ. Ở các bài sau, câu hỏi này sẽ cho ra ba trạng thái (Bài 3), hay cả một mảng trạng thái (Chương 3, 4).

A3. Công thức truy hồi



Hình 1.4. *Hai trường hợp của đỉnh v*

Lê Thành Nam

CÔNG THỨC

Với c chạy qua các con của v:

$f1[v] = a[v] + \Sigma\, f0[c]$; $f0[v] = \Sigma\, \max(f0[c], f1[c])$.

Nếu v là lá thì các tổng rỗng bằng 0: $f1[v] = a[v]$, $f0[v] = 0$.

Ý 1 Tập độc lập của T(v) tách thành các mảnh rời nhau

Bỏ đỉnh v đi, T(v) vỡ thành các cây con $T(c_1)$, $T(c_2)$, ..., $T(c_k)$, giữa chúng **không có cạnh nào**. Vì vậy một tập $S \subseteq T(v)$ là độc lập khi và chỉ khi: (1) mỗi mảnh $S \cap T(c_i)$ độc lập trong $T(c_i)$, và (2) không có cạnh nào giữa v và một con c_i mà cả hai đầu cùng thuộc S. Trọng số của S bằng ($a[v]$ nếu $v \in S$) cộng tổng trọng số các mảnh.

Ý 2 Trường hợp v ∈ S: được f1[v]

Điều kiện (2) buộc **mọi con $c_i \notin S$**. Ngoài ra các mảnh hoàn toàn độc lập với nhau, nên muốn tổng lớn nhất ta chọn **riêng từng mảnh** cho lớn nhất, với điều kiện c_i không được chọn: mảnh tốt nhất có trọng số $f0[c_i]$. Vậy $f1[v] = a[v] + \Sigma\, f0[c_i]$.

Ý 3 Trường hợp v ∉ S: được f0[v]

Điều kiện (2) luôn đúng, mỗi con c_i **tự do** được chọn hay không. Mảnh tốt nhất trong $T(c_i)$ là $\max(f0[c_i], f1[c_i])$, và lại chọn riêng từng mảnh. Vậy $f0[v] = \Sigma \max(f0[c_i], f1[c_i])$.

Ý 4 Vì sao được "chọn riêng từng mảnh"

Đây là **tính chất cấu trúc con tối ưu**: nếu trong lời giải tối ưu của T(v), mảnh trong $T(c_i)$ chưa tốt nhất, thay nó bằng mảnh tốt nhất (cùng trạng thái của c_i) thì các điều kiện (1), (2) vẫn đúng và tổng không giảm. Lí do sâu xa: các mảnh **chỉ "chạm" nhau qua đỉnh v**, và thông tin duy nhất cần ở chỗ chạm là c_i có thuộc S hay không, đã được ghi vào chỉ số 0/1 của trạng thái.

A4. Thứ tự tính: con trước cha, không đệ quy

Công thức của v dùng giá trị của các con, nên phải tính **mọi con trước cha**. Cách tự nhiên là DFS đệ quy, tính f khi ra khỏi đỉnh. Nhưng cây "đường thẳng" $2 \cdot 10^5$ đỉnh sẽ làm tràn ngăn xếp. Ta dùng mẹo đã quen từ Tập 1:

DUYỆT NGƯỢC THỨ TỰ BFS

1. BFS từ gốc 1, ghi lại thứ tự thăm ord[0..n − 1] và cha par[v].
2. Trong ord, cha luôn đứng **trước** con (cha được thăm trước khi đẩy con vào hàng đợi).
3. Duyệt ord **từ cuối về đầu**: khi tới v, mọi con của v đã được xử lí.

Còn một mẹo nữa giúp không cần danh sách con: khi xử lí xong u, ta **đẩy đóng góp của u lên cha** ngay lập tức: $f0[par[u]] \mathrel{+}= \max(f0[u], f1[u])$ và $f1[par[u]] \mathrel{+}= f0[u]$. Khi tới lượt v, các tổng Σ đã được cộng sẵn, chỉ còn cộng $a[v]$ vào $f1[v]$.

Lê Thành Nam

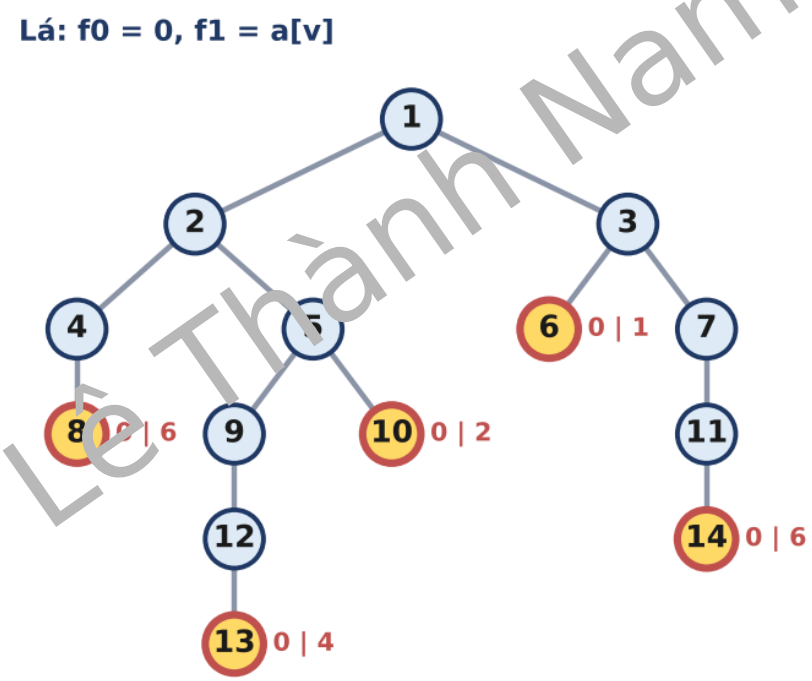
Code mẫu: QHĐ tập độc lập

```
for (int k = n - 1; k >= 0; k--) {
    int u = ord[k];
    f1[u] += a[u];
    if (u != 1) {
        int p = par[u];
        f0[p] += max(f0[u], f1[u]);
        f1[p] += f0[u];
    }
}
ans = max(f0[1], f1[1]);
```

A5. Chạy tay trên T14

Ta tính theo các "tầng từ dưới lên" để dễ theo dõi; mỗi đỉnh ghi cặp số **f0 | f1**. Đỉnh vừa tính tô vàng, đỉnh đã xong tô xanh.

Bước nhỏ 1 **Các lá**

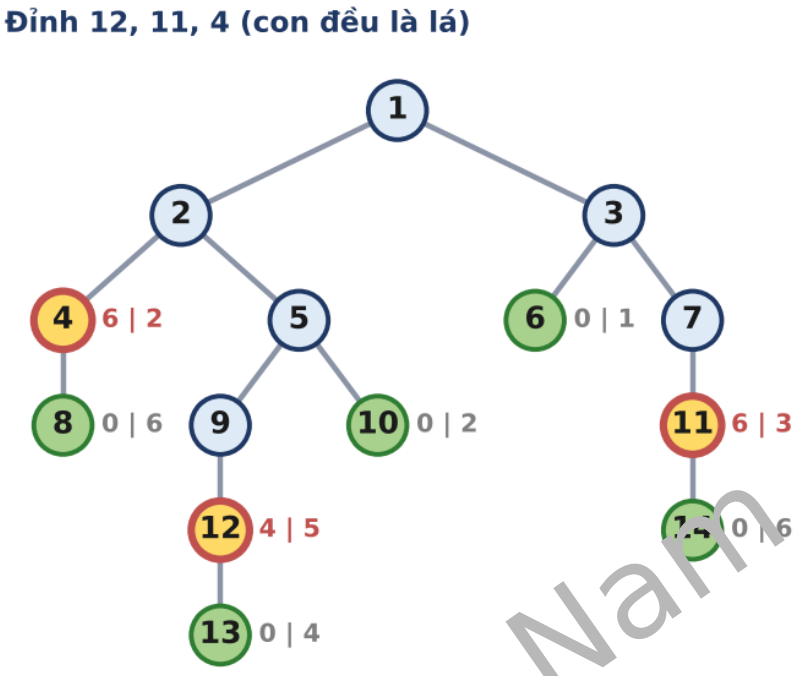


Lê Thành Nam

Hình 1.5. *Các lá*

Lá 13, 14, 8, 10, 6 có f0 = 0 và f1 = a[v]: 0 | 4, 0 | 6, 0 | 6, 0 | 2, 0 | 1.

Bước nhỏ 2 **Đỉnh 12, 11, 4**



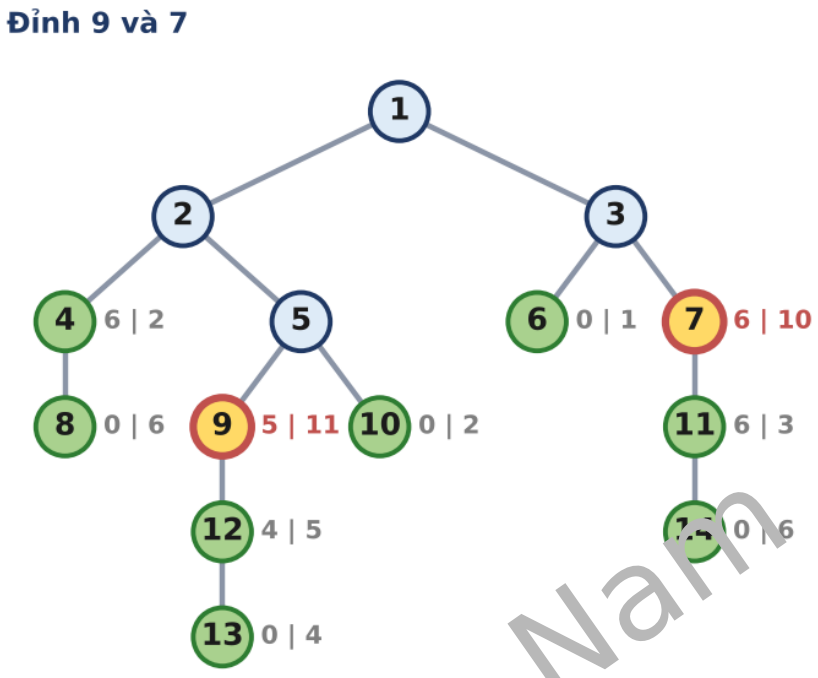
***Hình 1.6** Đỉnh 12, 11, 4*

Đỉnh 12 có một con 13: f1[12] = 5 + f0[13] = 5; f0[12] = max(0, 4) = 4.

Đỉnh 11 (con 14): f1 = 3 + 0 = 3; f0 = max(0, 6) = 6. Đỉnh 4 (con 8): f1 = 2; f0 = 6. Chú ý ở 11 và 4, **không chọn** lại tốt hơn: nhường chỗ cho lá nặng hơn.

Lê Thành Nam

Bước nhỏ 3 **Đỉnh 9 và 7**



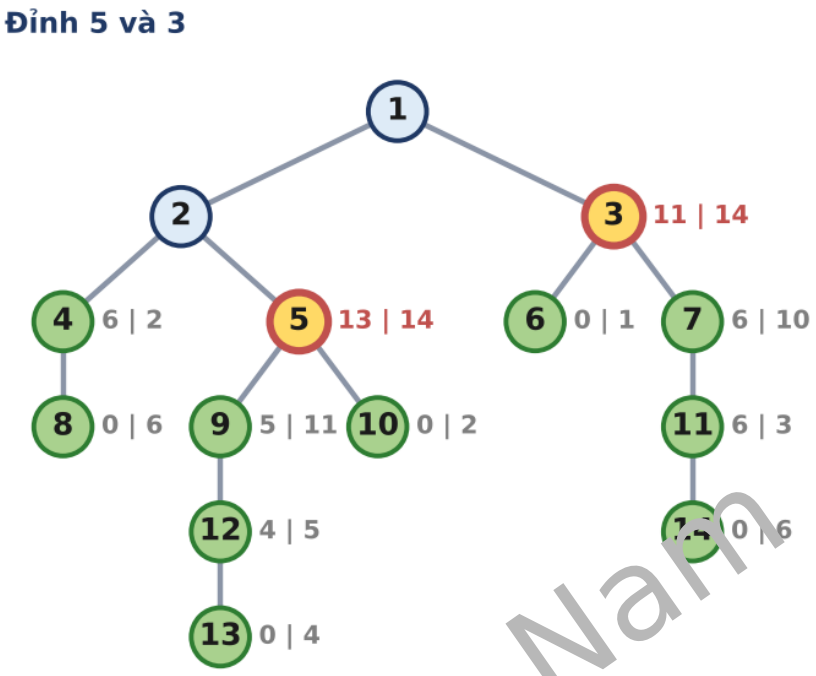
Hình 1.7. *Đỉnh 9 và 7*

f1[9] = 7 + f0[12] = 11; f0[9] = max(4, 5) = 5.

f1[7] = 4 + f0[11] = 10; f0[7] = max(6, 3) = 6.

Lê Thành Nam

Bước nhỏ 4 **Đỉnh 5 và 3**



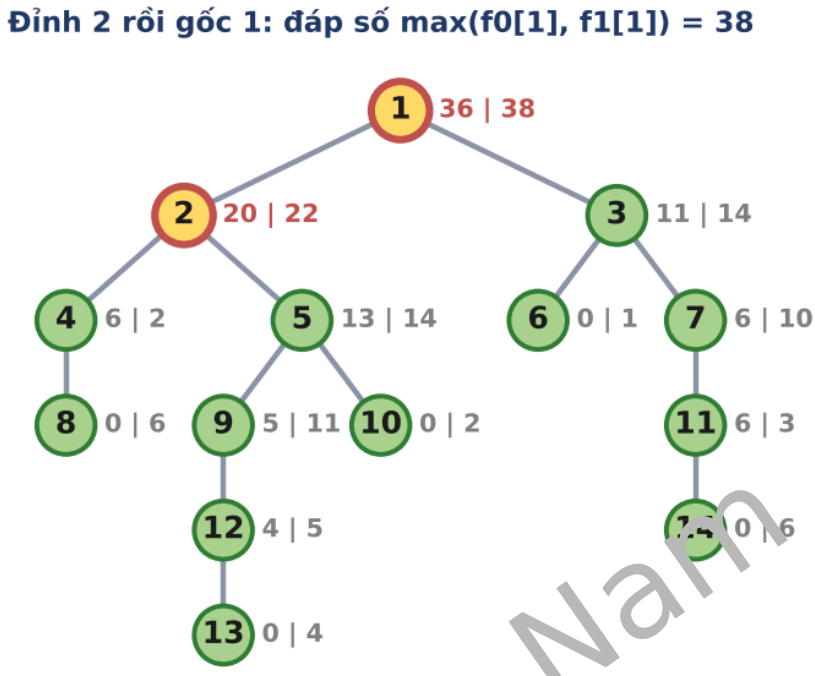
Hình 1.8. *Đỉnh 5 và 3*

Đỉnh 5 có hai con 9, 10: f1[5] = 9 + f0[9] + f0[10] = 9 + 5 + 0 = 14; f0[5] = max(5, 11) + max(0, 2) = 13.

Đỉnh 3 có hai con 6, 7: f1[3] = 8 + 0 + 6 = 14; f0[3] = max(0, 1) + max(6, 10) = 11.

Lê Thành Nam

Bước nhỏ 5 **Đỉnh 2 và gốc 1**



***Hình 1.9.** Đỉnh 2 và gốc 1*

f1[2] = 3 + f0[4] + f0[5] = 3 + 6 + 13 = 22; f0[2] = max(6, 2) + max(13, 14) = 20.

f1[1] = 7 + f0[2] + f0[3] = 7 + 20 + 11 = **38**; f0[1] = max(20, 22) + max(11, 14) = 36. Đáp số max(36, 38) = **38**.

v	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a[v]	7	[illegible]	8	2	9	1	4	6	7	2	3	5	4	6
f0[v]	36	20	11	6	13	0	6	0	5	0	6	4	0	0
f1[v]	38	22	14	2	14	1	10	6	11	2	3	5	4	6

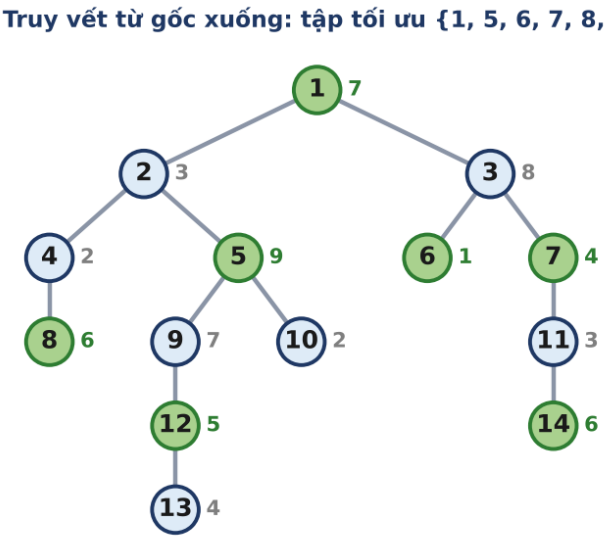
A6. Truy vết: in ra tập tối ưu

Bảng f chỉ cho **giá trị**. Muốn biết tập nào đạt giá trị đó, ta đi **từ gốc xuống**, mang theo một cờ "đỉnh này có được chọn không":

TRUY VẾT

1. Gốc 1 được chọn nếu f1[1] > f0[1] (ở đây 38 > 36: chọn).
2. Duyệt ord **từ đầu đến cuối** (cha trước con). Với mỗi đỉnh u ≠ 1: nếu cha của u đã được chọn thì u **không** được chọn; ngược lại u được chọn khi f1[u] > f0[u].

Lê Thành Nam



Hình 1.10. *Kết quả truy vết*

Kiểm tra: 7 + 9 + 1 + 4 + 6 + 5 + 6 = 38 và không có hai đỉnh xanh nào kề nhau. Truy vết đúng vì ở mỗi bước, ta chọn đúng nhánh của công thức đã cho giá trị tối ưu (khi cha được chọn chỉ có một nhánh; khi cha không được chọn, nhánh lớn hơn trong max).

A7. Khuôn 5 bước của QHĐ trên cây

Mọi bài trong Chương 1 và nhiều bài về sau đều được giải theo cùng một khuôn. Em nên tập viết ra đủ 5 bước trước khi gõ code.

KHUÔN 5 BƯỚC

1. **Treo cây** ở một gốc (thường là 1); bài toán trên T(v) là bài toán con.
2. **Trạng thái** f[v][s], với s là thông tin tối thiểu mà cha cần biết về v (v được chọn chưa, đã được phủ chưa, đã ghép chưa, ...).
3. **Công thức:** f[v][s] tính từ các f[c][·] của con; chứng minh bằng "bỏ v thì các cây con rời nhau".
4. **Thứ tự:** con trước cha (duyệt ngược BFS); cơ sở là lá.
5. **Đáp số** ở gốc, và **truy vết** nếu đề yêu cầu in lời giải.

ĐẾM SỐ PHÉP TÍNH

- Mỗi đỉnh u đóng góp O(1) phép tính lên cha: tổng **O(n)** thời gian, **O(n)** bộ nhớ.
- So với vét cạn 2^n tập con, đây là bước nhảy từ "chỉ chạy được $n \approx 25$" lên "chạy được $n \approx 10^6$".

Lê Thành Nam

BẪY THƯỜNG GẶP

- Đáp số có thể vượt 2^{31}: $n = 2 \cdot 10^5$ đỉnh, $a[v] \le 10^9$ cho tổng tới $2 \cdot 10^{14}$. Dùng **long long**.
- Quên cộng a[u] vào f1[u] **trước** khi đẩy f1[u] lên cha (code mẫu cộng ở dòng đầu tiên trong thân vòng lặp).
- Đệ quy DFS trên cây sâu: tràn ngăn xếp. Luôn dùng thứ tự BFS.
- In max(f0[1], f1[1]) chứ không phải f1[1]: gốc không bắt buộc được chọn.

GHI NHỚ

- $f1[v] = a[v] + \Sigma\, f0[c]$; $f0[v] = \Sigma \max(f0[c], f1[c])$; đáp số $\max(f0[1], f1[1])$.
- Duyệt ngược BFS và đẩy đóng góp lên cha: O(n), không đệ quy, không cần danh sách con.
- Trạng thái = điều cha cần biết về con.

PHẦN B BÀI VẬN DỤNG: TIỆC LIÊN HOAN

Áp dụng kiến thức Phần A để giải một bài toán theo khuôn đề thi [TIECLH].

BÀI TOÁN. TIỆC LIÊN HOAN [TIECLH]

Nhân ngày Nhà giáo Việt Nam, trường tổ chức tiệc liên hoan. Trường có n thầy cô, đánh số từ 1 đến n; thầy hiệu trưởng là người 1. Mỗi người i ($2 \le i \le n$) có đúng một **cấp trên trực tiếp** p_i. Mỗi người i có độ vui a_i.

Để mọi người thoải mái, ban tổ chức **không mời cùng lúc một người và cấp trên trực tiếp của người đó**. Hãy chọn danh sách khách mời có tổng độ vui lớn nhất.

Dữ liệu vào:

- Dòng đầu chứa số nguyên n ($1 \le n \le 2 \cdot 10^5$).
- Dòng thứ hai chứa n số nguyên $a_1, a_2, \ldots, a_n$ ($0 \le a_i \le 10^9$).
- Dòng thứ ba chứa $n - 1$ số nguyên $p_2, \ldots, p_n$; dữ liệu bảo đảm từ mọi người đi theo cấp trên đều tới hiệu trưởng.

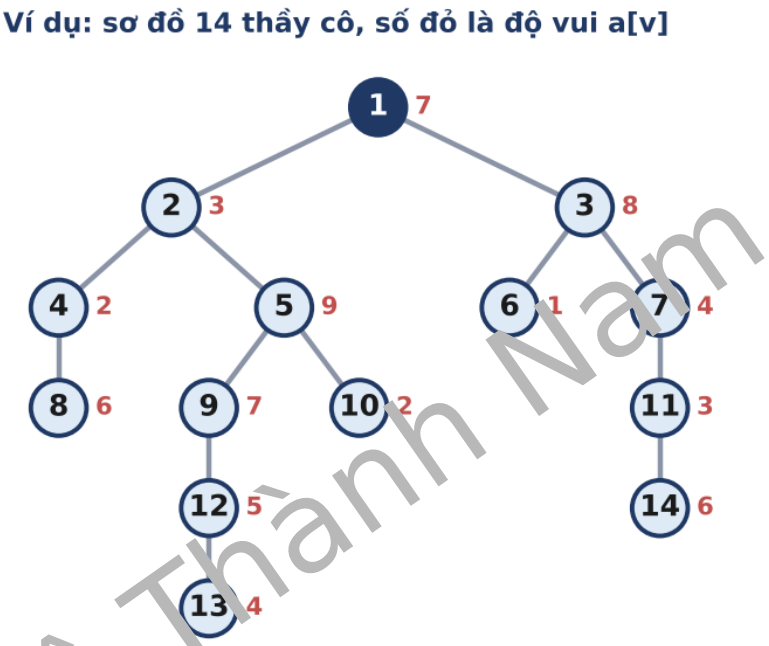
Kết quả:

- Một số nguyên là tổng độ vui lớn nhất.

Tên tệp: TIECLH.INP, TIECLH.OUT. Thời gian: 1 giây. (Bài kinh điển "Anniversary party", ở dạng có trọng số.)

Lê Thành Nam

Dữ liệu vào	Kết quả	Giải thích
14 7 3 8 2 9 1 4 6 7 2 3 5 4 6 1 1 2 2 3 3 4 5 5 7 9 12 11	38	Sơ đồ là cây T14 ở Hình 1.11. Mời 1, 5, 6, 7, 8, 12, 14.
3 5 1 5 1 2	10	Đường 1 – 2 – 3: mời 1 và 3.



Hình 1.11. *Sơ đồ trong ví dụ 1*

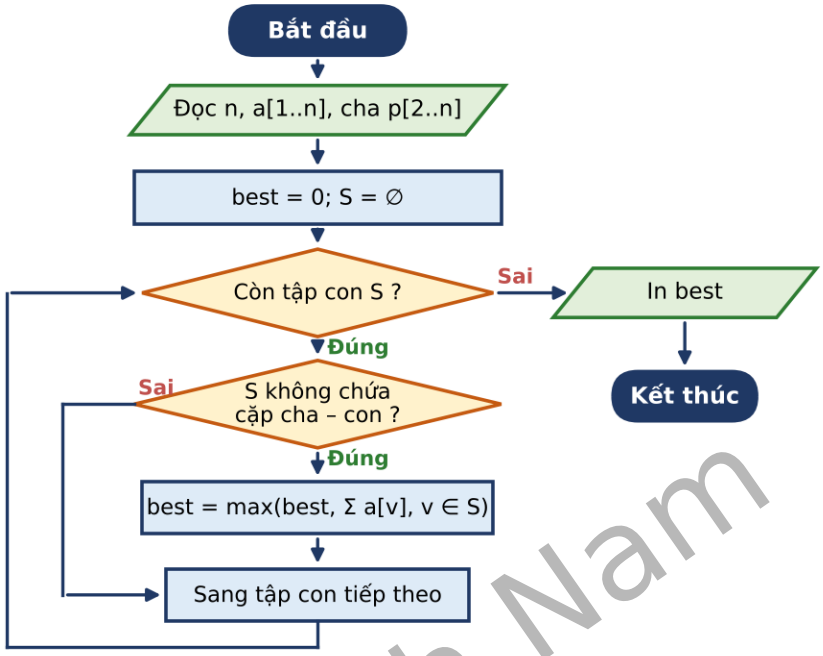
Lê Thành Nam

Dịch sang Phần A: "người và cấp trên trực tiếp" là một cặp **cha – con** trên cây; danh sách khách mời là một **tập độc lập**, độ vui là trọng số.

B1. Ý tưởng ngây thơ: Thử mọi danh sách

Có 2^n cách chọn tập khách mời. Với mỗi tập S (biểu diễn bằng một số nguyên mask, bit i − 1 bằng 1 nghĩa là mời người i), kiểm tra không có người nào được mời cùng cấp trên trực tiếp, rồi cập nhật tổng lớn nhất.

Sơ đồ khối

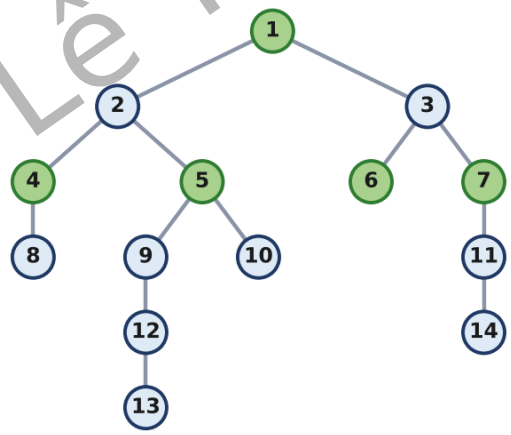


Hình 1.12. *Sơ đồ khối của cách thử mọi tập con*

Chạy tay

Bước nhỏ 1 **Một tập con hợp lệ**

Vét cạn: thử mọi tập con (2^14 = 16384 tập). Ví dụ tập này độc lập, tổng 23



Hình 1.13. *Một tập con hợp lệ*

Tập {1, 4, 5, 6, 7} không chứa cặp cha – con nào, tổng 7 + 2 + 9 + 1 + 4 = 23 < 38. Phải thử hết 16 384 tập mới biết 38 là lớn nhất.

Lê Thành Nam

Chương trình ngây thơ

TIECLH_TRAU.CPP: Thử mọi tập con, O(2^n · n)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(false);
6      cin.tie(nullptr);
7      if (fopen("TIECLH.INP", "r")) {
8          freopen("TIECLH.INP", "r", stdin);
9          freopen("TIECLH.ANS", "w", stdout);
10     }
11     int n;
12     cin >> n;
13     vector<long long> a(n + 1);
14     for (int i = 1; i <= n; i++) cin >> a[i];
15     vector<int> par(n + 1, 0);
16     for (int i = 2; i <= n; i++) cin >> par[i];
17     long long best = 0;
18     for (int mask = 0; mask < (1 << n); mask++) {
19         bool ok = true;
20         long long s = 0;
21         for (int i = 1; i <= n && ok; i++)
22             if (mask >> (i - 1) & 1) {
23                 s += a[i];
24                 if (i > 1 && (mask >> (par[i] - 1) & 1) == 0) ok = false;
25             }
26         if (ok) best = max(best, s);
27     }
28     cout << best << '\n';
29     return 0;
30 }
```

- **Dòng 18:** duyệt mọi mask từ 0 đến $2^n - 1$.
- **Dòng 21–25:** với mỗi người i được mời, cộng a_i; nếu cấp trên p_i cũng được mời thì tập không hợp lệ.

ĐẾM SỐ PHÉP TÍNH

- $2^n \cdot n$ phép: n = 20 đã là $2 \cdot 10^7$; n = 40 là $4 \cdot 10^{13}$, không thể chạy.

BẾ TẮC

Vét cạn xét lại cùng một cây con rất nhiều lần với những lựa chọn **không ảnh hưởng gì** đến phần còn lại. Thực ra phần còn lại của cây chỉ cần biết **một bit** về mỗi cây con T(v): v có được mời hay không. Đó chính là trạng thái QHĐ.

B2. Phá bế tắc: QHĐ hai trạng thái

Bước 1 Đọc cây, lập thứ tự BFS

Dữ liệu cho sẵn cấp trên p_i, nhưng **không bảo đảm $p_i < i$**, nên vẫn phải BFS từ người 1 theo danh sách con để có thứ tự cha trước con.

Lê Thành Nam

Bước 2 **Tính f0, f1 từ dưới lên**

Theo đúng mục A4–A5; bảng giá trị của ví dụ 1 là bảng ở mục A5, đáp số 38.

Bước 3 **In đáp số**

max(f0[1], f1[1]). Với ví dụ 2: f0 | f1 của 3, 2, 1 lần lượt là 0 | 5, 5 | 1, 5 | 10, đáp số 10.

> **ĐẾM SỐ PHÉP TÍNH**
>
> - O(n) thời gian và bộ nhớ; n = $2 \cdot 10^5$ chạy dưới 0,05 giây (đã đo với cây ngẫu nhiên và cây đường thẳng).

B3. Chương trình hoàn chỉnh

TIECLH.CPP: QHĐ tập độc lập, O(n)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TIECLH.INP", "r")) {
        freopen("TIECLH.INP", "r", stdin);
        freopen("TIECLH.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<int> par(n + 1, 0);
    vector<vector<int>> ch(n + 1);
    for (int i = 2; i <= n; i++) {
        cin >> par[i];
        ch[par[i]].push_back(i);
    }
    vector<int> ord;
    ord.push_back(1);
    for (int k = 0; k < (int)ord.size(); k++)
        for (int v : ch[ord[k]]) ord.push_back(v);
    vector<long long> f0(n + 1, 0), f1(n + 1, 0);
    for (int k = n - 1; k >= 0; k--) {
        int u = ord[k];
        f1[u] += a[u];
        if (u != 1) {
            int p = par[u];
            f0[p] += max(f0[u], f1[u]);
            f1[p] += f0[u];
        }
    }
    cout << max(f0[1], f1[1]) << '\n';
    return 0;
}
```

Lê Thành Nam

- **Dòng 15–20:** đọc cấp trên, lập danh sách con ch.
- **Dòng 21–24:** BFS bằng chính vector ord (vừa là hàng đợi vừa là kết quả).
- **Dòng 26–34:** duyệt ngược ord; cộng a[u] vào f1[u], rồi đẩy đóng góp của u lên cha.

- **Dòng 35:** đáp số ở gốc.

B4. Mở rộng

Mở rộng 1: In danh sách khách mời

Truy vết như mục A6. Nếu có nhiều đáp án, quy ước "chọn khi f1 > f0" cho một đáp án cụ thể; để có thể yêu cầu ưu tiên ít người nhất, khi đó so sánh thêm số người (lưu cặp (tổng, −số người)).

Mở rộng 2: Đếm số tập độc lập

Thay max bằng cộng và cộng bằng nhân: c1[v] = Π c0[c], c0[v] = Π (c0[c] + c1[c]); số tập độc lập (kể cả tập rỗng) là c0[1] + c1[1], lấy theo modulo. Với đường 3 đỉnh ta được 5 tập: ∅, {1}, {2}, {3}, {1, 3}.

Mở rộng 3: Ràng buộc "không mời cả người và cấp trên thứ 2"

Khi ràng buộc trải tới ông bà, cha cần biết thêm thông tin về cháu trạng thái phải mở rộng (chẳng hạn ghi cả việc có con nào được chọn hay không). Đây là bài luyện tập tốt cho câu hỏi "cha cần biết gì".

LUYỆN TẬP

Bài	Tên	Gợi ý
1.1	ANNIVERSARY PARTY	Bài kinh điển (HDU 1520, Timus 1039): nộp đúng chương trình Phần E.
1.2	ĐẾM TẬP ĐỘC LẬP	Mở rộng 2 (AtCoder DP Contest P – Independent Set, tô hai màu trắng đen).
1.3	IN KHÁCH MỜI	Mở rộng 1, in danh sách tăng dần.

Tự kiểm tra nhanh:

1. Với T14, nếu bắt buộc mời hiệu trưởng thì đáp số là bao nhiêu? *(f1[1] = 38.)*
2. Nếu bắt buộc không mời hiệu trưởng? *(f0[1] = 36.)*
3. Vì sao trên cây chỉ cần hai trạng thái, còn trên đồ thị tổng quát bài toán là NP-khó? *(Trên cây, bỏ một đỉnh thì các phần rời nhau, chỉ "chạm" nhau qua đúng đỉnh đó.)*

Lê Thành Nam